

Task Assessment – Software Engineer Intern

Overview

Build a simple full-stack app with user authentication and user CRUD. That's it - keep it minimal and functional.

Tech Stack (Required)

Layer	Technology
-------	------------

Frontend	Next.js
----------	---------

Backend	NestJS
---------	--------

Database	MongoDB (Mongoose)
----------	--------------------

Styling	Tailwind CSS
---------	--------------

Auth	JWT stored in cookies
------	-----------------------

Requirements

1. Authentication

- Register - name, email, password (hash password with bcrypt)
- Login - issue a JWT and set it as an HTTP-only cookie
- Logout - clears the cookie
- Protect authenticated routes using a Guard (NestJS) and simple middleware/session check (Next.js)

2. User CRUD

Once logged in:

- View list of users
- View a single user
- Update own profile (name/email)
- Delete own account

Keep it simple - no roles/permissions needed, just standard CRUD scoped to the logged-in user.

3. Frontend (Next.js)

- Pages: /register, /login, /dashboard (list + profile view)
- Tailwind CSS for basic, clean styling
- Simple form validation and error messages

4. Backend (NestJS)

- auth and users modules
- DTO validation with class-validator
- Mongoose schema for User (name, email unique, hashed password)

Deployment (Required)

- Frontend → Vercel
- Backend → Render
- The deployed frontend must call the live Render backend URL (not localhost) via an environment variable (e.g. NEXT_PUBLIC_API_URL)
- Cookies must work correctly cross-domain in production (set SameSite, Secure, and CORS with credentials: true properly between the Vercel and Render domains)
- The full flow (register → login → CRUD → logout) must work end-to-end on the live deployed links - not just locally. This will be tested directly.
- Share both live links plus the GitHub repo

Submission

1. Public GitHub repo with a short README (setup steps + .env.example)
 2. Deployed frontend (Vercel) and backend (Render) links
 3. Send everything by [insert deadline here]
-

Evaluation Criteria

Area	What we're looking for
Auth correctness	JWT + cookie flow works end-to-end
Security basics	Hashed passwords, HTTP-only cookies, no leaked data
CRUD correctness	Users can view/update/delete correctly
Code quality	Clean, readable, reasonably organized
Deployment	Both links actually work